



International Islamic University Chittagong

Computer Science and Engineering

Assignment – 1

Course Code: CSE-2422

Course Title: Algorithms Lab

Submitted to:

Mr. Mehrajul Islam
Adjunct Lecturer, CSE
International Islamic University Chittagong

Submitted by:

Name : Tofayel Ahmmed
Student ID : C-243236
Program : B.Sc. in Computer Science and Engineering
Batch : CSE-58
Section : 4FM
Semester : 4th (Fourth)

Submission Date: March 28, 2026

CSE-2422 Algorithms: Sorting Assignment Report

Student Name: Tofayel Ahmmed

Student ID: C243236

Course: CSE-2422 Algorithms

GitHub Link: [Algorithm](#)

1. Algorithm Description

1. Insertion Sort

Insertion Sort is an incremental algorithm. It builds the final sorted array one element at a time by picking a "key" from the unsorted portion and inserting it into its correct position within the already-sorted left portion.

- **Analogy:** Like sorting a hand of playing cards.
- **Strength:** Extremely efficient for small datasets ($N < 50$) or nearly-sorted data.

Weakness: Its quadratic nature makes it impractical for large, random datasets. 2.

Merge Sort

Merge Sort is a quintessential **Divide and Conquer** algorithm. It recursively bisects the array into halves until it reaches single-element sub-arrays, then merges them back together in sorted order.

- **Strategy:** The "Merge" step is where the work happens, comparing elements from two sorted lists to create a single sorted output.
- **Consistency:** It is highly predictable, maintaining $\Theta(n \log n)$ performance regardless of the initial data distribution.

3. Quick Sort

Quick Sort also uses **Divide and Conquer** but focuses on **Partitioning**. It selects a "pivot" element and rearranges the array so that smaller elements are to the left and larger ones are to the right.

- **Implementation:** This version uses a **randomized pivot**, which is a critical engineering choice. It prevents the $\Theta(n^2)$ worst-case scenario that occurs with already-sorted data.
- **Efficiency:** Known for excellent cache locality and low memory overhead since it sorts "in place."

2. Experimental Results

Measurements were taken using `chrono::high_resolution_clock` with `-O2` optimization. Values recorded as `0.00000000s` indicate sub-microsecond execution times.

Algorithms	N=10	N=100	N=1000	N=10 ⁴	N=10 ⁵
Insertion Sort	0.00000000 s	0.00000000 s	0.00000000 s	0.01205500 s	1.14867300 s
Merge Sort	0.00000000 s	0.00000000 s	0.00000000 s	0.00301300 s	0.01804000 s
Quick Sort	0.00000000 s	0.00000000 s	0.00000000 s	0.00104800 s	0.00605400 s

3. Time Complexity Analysis

Algorithms	Best Case	Average Case	Worst Case
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$

Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

4. Discussion

1. Why some algorithms performed faster than others

The primary reason for the performance gap is the difference in Time Complexity classes.

- Insertion Sort utilizes a nested loop structure, resulting in an average and worst-case complexity of $\Theta(n^2)$. As the input size increases, the number of comparisons and shifts grows quadratically, making it very slow for large datasets like 10^5 .
- Merge Sort and Quick Sort are built on the Divide and Conquer philosophy. By recursively breaking the problem into smaller sub-problems, they achieve an average complexity of $\Theta(n \log n)$.
- Quick Sort typically outperformed Merge Sort in the experiment (0.006 s vs 0.018 s for $N=10^5$) because it sorts in-place. This results in better cache locality and eliminates the overhead of allocating and merging temporary arrays that Merge Sort requires.

2. How the experimental results and runtime growth relate to theoretical complexity

The empirical data from the experiment provides a clear validation of Big- Θ theory:

- Insertion Sort: When the input size increased from 10,000 to 100,000 (10 times increase), the execution time grew from 0.012 s to 1.148 s. This is a growth factor of approximately 95 times, which closely matches the theoretical $\Theta(n^2)$ prediction that 10 times increase in N leads to a $100\times$ (10^2) increase in time.
- Merge and Quick Sort: For the same 10 times increase in input size, these algorithms showed a much smaller increase in runtime (roughly 6 times for Merge Sort). This confirms that their growth rate is near-linear ($\Theta(n \log n)$), rather than quadratic.

3. Interesting observations noticed during the experiment

- The Threshold of Significance: For small input sizes ($N=10, 100, 1000$), the execution times were recorded as 0.00000000 s. This indicates that at small scales, the constant-time overhead of the system and the efficiency of the CPU make the difference between $O(n^2)$ and $O(n \log n)$ negligible.
- Randomized Pivot Efficiency: The Quick Sort implementation used a randomized pivot. This is an important observation because it ensures that the algorithm avoids the $\Theta(n^2)$ worst-case scenario often triggered by already-sorted data, maintaining high performance even on various dataset distributions.
- Compiler Optimization: Using the `-O2` flag likely contributed to the extremely fast execution times for Merge and Quick Sort, as the compiler can optimize

recursive calls and memory access patterns more effectively for these algorithms than for the simple loops of Insertion Sort.